Discriminating UAV from non-UAVs, based on Time-series Location Data

Kiliemah Ouattara, Alon Kipnis

Abstract—In this paper, we address the problem of discriminating Unmanned Aerial Vehicles (UAVs) from other objects using azimuth and elevation measurements. Our approach explores various techniques for processing the time-series data, comparing the effectiveness of feature extraction prior to detection using XGBoost with that of sequence classification via Transformers-based neural networks. Both methods yield promising results, enabling efficient, accurate and fast UAV detection. Furthermore, in-depth data analysis through freaAI algorithm reveals that sequence classification neural networks perform better than XGboost when encountering unfavorable data slices.

Keywords—Machine Learning, Time series, Transformer, UAV detection

I. Introduction

AS Unmanned Aerial Vehicles (UAVs) increasingly pose security threats, counter-UAV systems have become critical for preventing potential damage. However, UAV detection remains an evolving field, with numerous challenges emerging over time. A key priority is to achieve accurate results by minimizing false positives and ensuring that counter-UAV systems are not misled. Additionally, detection speed is a crucial factor when evaluating the effectiveness of UAV detection systems.

Sensors based on radio signals, such as RADAR, are capable of detecting flying objects at greater distances compared to optical devices, but they typically only provide information about an object's location in the air. Therefore, it becomes essential to determine whether the detected object is a UAV. As UAVs vary increasingly in size, maneuverability, and noise emissions, effective track classification, an area that remains understudied, is vital [7].

In this work, we explore UAV detection using multi-dimensional time-series derived from azimuth and elevation measurements. Data are collected using AirScout, a Electro-Optical / Infra-Red (EO/IR) system which uses AI-driven software to power optical tracking of UAV. Starting with the construction of various dataset settings, we employ both feature extraction prior to models detection and sequence classification neural networks. In the first approach we compute features and store them, before feeding the model. The second approach uses neural network to compute the feature and output predictions. Then, we assess the performance of these detectors across different settings and evaluate their effectiveness on more complex flight tracks.

Kiliemah Ouattara and Alon Kipnis are with the Efi Arazi School of Computer Science, Reichman University, Herzliya, Israel (Corresponding author, e-mail: ouattara.nadienkolo@post.runi.ac.il).

II. Related work

Radar track data have emerged as a valuable resource in distinguishing UAVs from non-UAVs, aiming to minimize false positives while enhancing UAV detection [7] . Previous methodologies predominantly relied on Micro-Doppler analysis [6] [5], which, although effective for close-range detection of metal-rotor UAVs with strong radar echoes [2], proved inadequate for long-range applications.

Transitioning from Micro-Doppler to radar track analysis preserves the benefits of feature extraction from track data [4], akin to Micro-Doppler analysis, while extending the detection range [7].

In a notable recent study [7], researchers leveraged features measuring the mean speed, the linearity of the motion and the mean of the peak amplitude of the power extracted from radar tracks to construct a decision tree for improved interpretability. These features, previously identified as highly effective by Mohajerin et al. in 2014 [4], proved instrumental in classification tasks up to a range of 4 km, as demonstrated by Messina et al. in 2019 [3]. Selecting a subset of features from the 50 extracted, Messina et al. employed a SVM model to achieve discrimination. Furthermore, Chen et al. in 2019 [2] used a motion model to estimate target maneuverability (conversion frequency) and used a threshold-based approach, exploiting the fact that birds typically exhibit higher maneuverability compared to UAVs.

Although feature extraction methods and motion models have exhibited promising accuracy over longer ranges while mitigating false positives, recent advances in machine learning such as transformer-based [8] techniques also appear promising but have not yet been explored.

III. Method

This work aims to construct detectors using two distinct techniques. The first approach, feature-based, involves extracting features from multidimensional time series data and subsequently discriminating based on these features. Three models will be employed to implement this technique: a decision tree-based model and a linear classifier.

The second technique adopts a sequence-based approach, utilizing multi-dimensional time series directly to capture essential discriminatory information. To accomplish this, a transformer model will be used. Subsequently, we conclude with a comparative analysis based on specific metrics to evaluate the effectiveness of each approach.

Furthermore, training data pre-processing is also crucial. Different approaches have been used in the problem. One

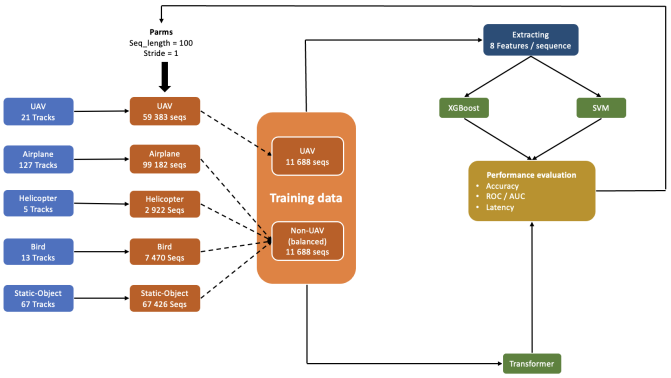

Fig. 1: Data Processing Architecture. Non-UAV samples are coming from airplanes, helicopters, birds and static objects. The dataset is constructed so that all 4 non-UAV are balanced inside non-UAV label. Additionally the resulting dataset is balanced between UAV and non-UAV.

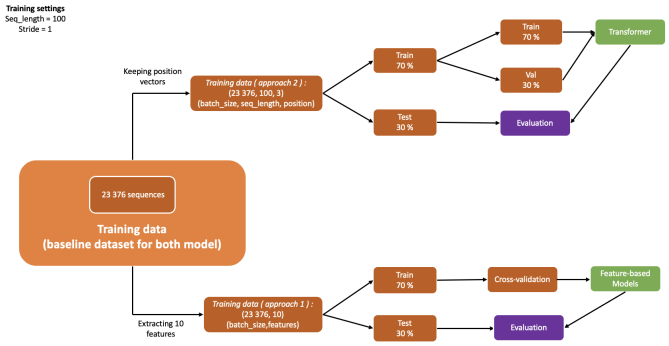

Fig. 2: Training and testing pipeline. From a baseline balanced dataset, 10 features are extracted on each sequence, to train the first approach models. Then the train/test splits are constructed, in a way that each training sequence does not have any overlapping positions with a given testing sequence. This ensure fairness in the evaluation step.

consisting of smoothing the track with waypoints achieves a good result in classification using feature extraction, before model prediction by Mohajerin et al. in 2014 [4]. Another one using a non-smoothed track seems to make discrimination harder because of variations, according to Tahmoush, D. (2023, June) [7].

In this study, we are comparing on the second approach, which causes more trouble to feature-based models.

A. Data processing

The architecture in Figure 1 represents the comprehensive data flow process, encompassing the journey from track records to the evaluation of model performance.

Key parameters driving the experimental process include sequence length and strides. Within each track, a series of sequences are cropped. The sequence length dictates the duration of each crop, while the stride determines the transition to another crop. The significance of these two parameters lies in their respective roles: the sequence length facilitates capturing latency before detection, while the stride influences the variance of training samples, thereby impacting testing outcomes. A smaller stride tends to yield closer samples, whereas a larger stride introduces greater variability.

Moreover, to ensure a fair comparison, hyperparameters and architecture of the models remain consistent throughout the experiment, without fine-tuning.

B. Training on extracted features

From the baseline dataset obtained through sequence length and stride parameters, 10 features are calculated on each sequence (Figure.2). Using the resulting feature dataset, an XGboost and SVM models are fitted. Table 1 describes the extracted features.

C. Training on position sequences

Concerning the transformer, it uses a single standard encoding layer, to extract features and one fully connected layer for classification (Figure.4).

For a given ”batch” containing one sample with shape (1, 100, 3) as (batch size, sequence length, position vector), the input positions undergo feature extraction, via a fully connected embedding layer, resulting in a reshaped tensor of dimensions (1, 100, 16). Subsequently, positional encoding is applied to this feature vector. Following this, a single transformer encoding layer with two attention heads and a feed-forward layer of dimensionality 2048 is employed. A causal mask is applied after feature extraction, to only use previous positions while making prediction. Upon normalization, the encoding layer outputs a feature vector of shape (1, 100, 16).

For the classification task, k = seq-length, fully connected layers are used to compute class scores for each position. These scores are summed up to calculate the whole sequence scores, which is then subjected to a softmax function to obtain ”probabilities”. Subsequently, the binary cross-entropy loss is computed to train the model, quantifying how well the network’s soft prediction aligns with the actual discrete class indicator label vector.

D. Train/test splitting

The splitting algorithm between train and test set is presented in Figure 3. While building a sequence dataset using a track, we make sure that there is no overlaps between sample in train set and those in test set. This result in a fair evaluation process, as we can be sure there is no portion of sequence, already seen in the training step, when we are testing.

A skip is implemented in the sequence construction, to break overlaps. After each break, sequences are flagged to build a disjoint partition. Every overlapping samples are stuck in the same set, so that they can’t be in both train and test set. Notice that it will leads to approximately equal number of train samples and test samples. As we want more training samples, we solve this by merging some test partition (overlapping samples) into the train set.

IV. Experimentation

The experimentation process assess both approach (feature dataset, sequences data) performances based on training data

Feature	Description	Equation
f_1_zx	Mean of slope difference between consecutive positions in plane zx	$\text{f\_1\_zx} = \frac{1}{n}\sum_{i=0}^{n}(l_{i+1} - l_i), \quad l_i = \tan^{-1}\left(\frac{z_{i+1} - z_i}{x_{i+1} - x_i}\right)$ (1)
f_1_yx	Mean of slope difference between consecutive positions in plane yx	$\text{f\_1\_yx} = \frac{1}{n}\sum_{i=0}^{n}(l_{i+1} - l_i), \quad l_i = \tan^{-1}\left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i}\right)$ (2)
f_2_zx	Variance of slope difference in plane zx	$\text{f\_2\_zx} = \frac{1}{n-1}\sum_{i=0}^{n}((l_{i+1} - l_i) - \text{f\_1\_zx})^2$ (3)
f_2_yx	Variance of slope difference in plane yx	$\text{f\_2\_yx} = \frac{1}{n-1}\sum_{i=0}^{n}((l_{i+1} - l_i) - \text{f\_1\_yx})^2$ (4)
f_3	Max distance between consecutive positions	$\text{f\_3} = \max(d(x_i, x_{i+1}))$ (5)
f_4	Ratio of min to max distance	$\text{f\_4} = \frac{\min(d(x_i, x_{i+1}))}{\max(d(x_i, x_{i+1}))}$ (6)
f_5	Total traverse distance	$\text{f\_5} = \sum_{i=0}^{n-1} d(x_i, x_{i+1})$ (7)
f_6	Mean velocity	$\text{f\_6} = \frac{1}{n}\sum_{i=0}^{n} v_i$ (8)
f_7	Mean acceleration	$\text{f\_7} = \frac{1}{n-1}\sum_{i=0}^{n-1} a_i$ (9)
f_8	Acceleration variance	$\text{f\_8} = \frac{1}{n-1}\sum_{i=0}^{n-1}(a_i - \text{f\_7})^2$ (10)

TABLE I: Feature descriptions and equations. Feature efficiencies in the UAV detection problem have been demonstrated by Mohajerin et al. [4].

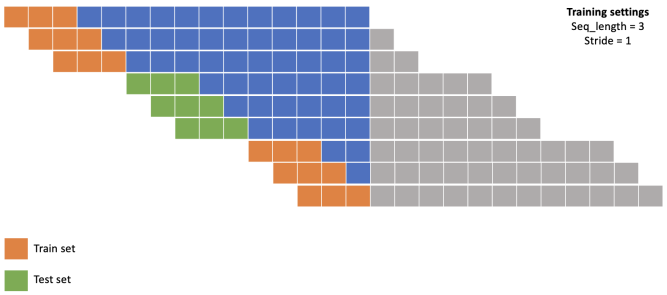

Fig. 3: Train/test split. While going over a track, the striding process is interrupted at certain points to create disjoint partitions.

features	node id	f1	f1 imp	precision	recall	size	proportion	min val1	max val1	min val2	max val2
f_1_yx	21	0.807692	0.004788	0.851351	0.797468	145	5.677369	-0.011161	0.002238	-1000	inf
f_1_zx	16	0.90573	0.002148	0.837607	0.924528	186	7.282694	0.035502	0.296373	-1000	inf
f_1_zx-f_1_yx	17	0.807692	0.012614	0.851351	0.797468	145	5.677369	-1000	inf	-0.011161	0.002238
f_1_zx-f_2_zx	21	0	0.006857	0	0	1	0.039154	0.3117	inf	-0.02413	-0.022846

TABLE II: XGBoost 4 bad data slices. The table represents 4 data slices, on which the XGBoost model performs the worst. Each data slice is represented by the min and max value inside the slice, for each feature. The proportion is the amount of samples, representing by this slice in the total amount of test samples. The table displays the f1 score on the specific slices and the potential improvement of the overall f1 (f1 imp) if these slices had overall f1-score.

setting (sequence length, stride), detection delay and worse data slices.

A. Training data setting

Figures 5 and 6 present the F1 and AUC performance metrics for each model across different dataset configurations. A True detection (or True positive) refers to an UAV effectively classify as UAV. Overall, the Transformer model consistently achieves an F1 score around 95%. XGBoost also demonstrates steady results, with an average F1 score around 90%, though it is consistently outperformed by the Transformer. On the other hand, SVM shows significantly lower performance on the selected features. Notably, both of the Transformer and XGBoost display strong results on the (5,2) training data configuration, highlighting their potential in this specific setting (Figure 6).

Additionally the error analysis (Figure 7) reveals that the transformer do not confuse static-object and helicopter as UAV, while other models make this confusion. Secondly, the worst models, SVM struggle the most with static-objects. Finally the best models (XGB and Transformer) are perspectively challenged by airplanes, followed by birds.

B. Detection latency

Figures 8 and 9 illustrate the models' performance across different delay intervals. Both the AUC and F1 scores for the Transformer and XGBoost show a slight improvement as the delay decreases. While this trend may also be linked to the increased number of training samples, the overall performance remains high for both models, with the Transformer consistently outperforming XGBoost.

C. Performances on data slices

In this section, we focus on analyzing the Transformer and XGBoost models on the (sequence-length = 5, stride = 2,

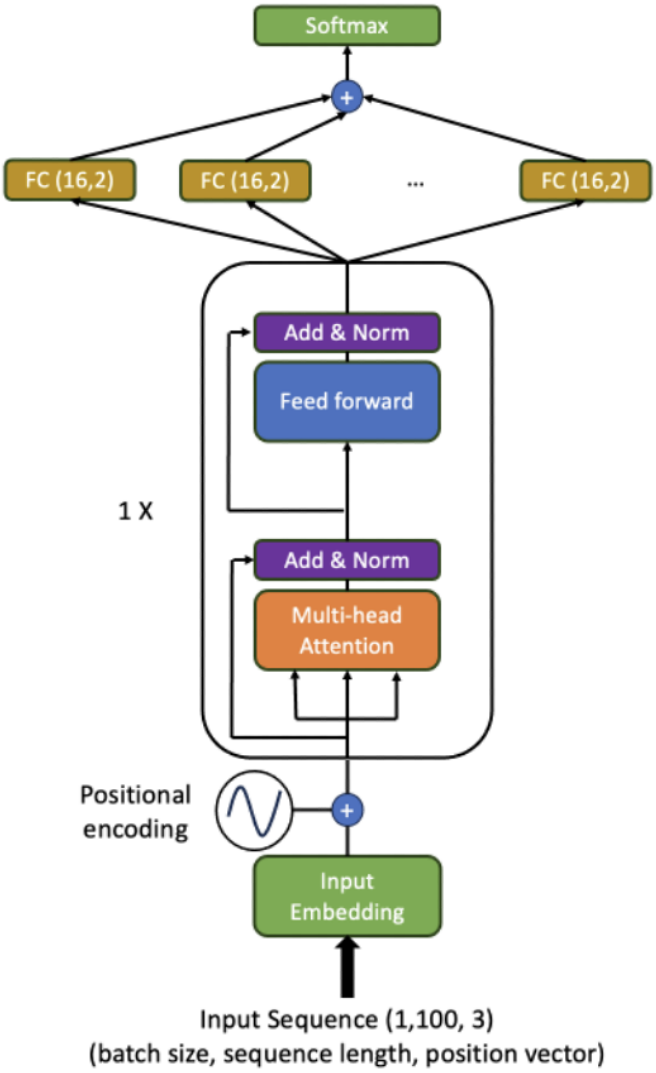

Fig. 4: Transformer Encoder architecture. For each input sequence, the classifier score is obtained by aggregating local (specific position in the input sequence) scores.

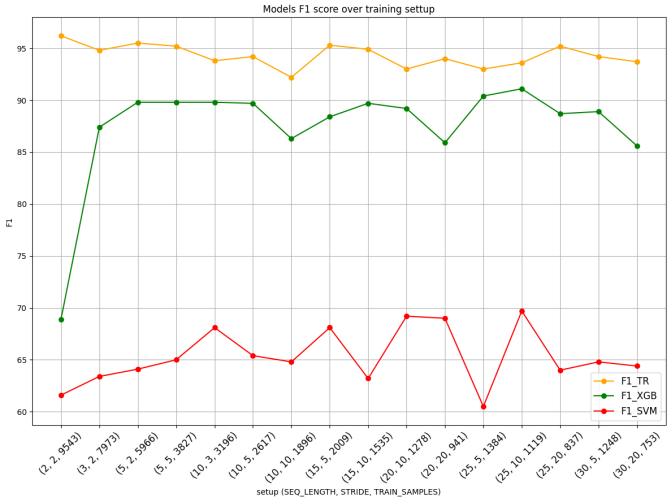

Fig. 5: F1 score over training setting. F1-TR, F1-XGB and F1-SVM, respectively represent F1-scores of Transformer, XGBoost and SVM models over different training settings.

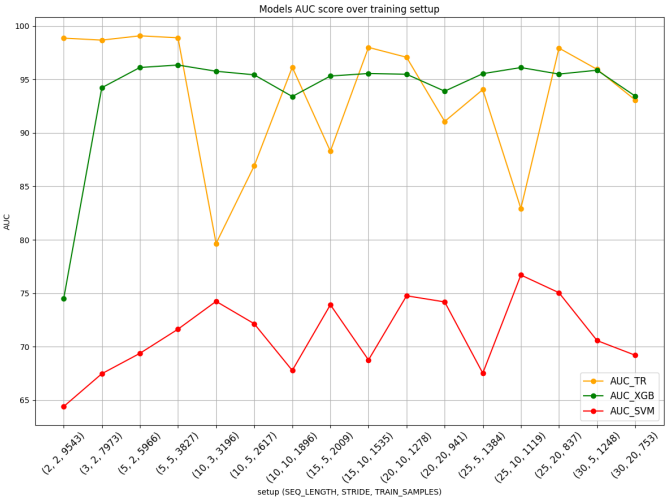

Fig. 6: AUC over training setting. AUC-TR, AUC-XGB and AUC-SVM respectively represent AUC of Transformer, XGBoost and SVM models over different training settings.

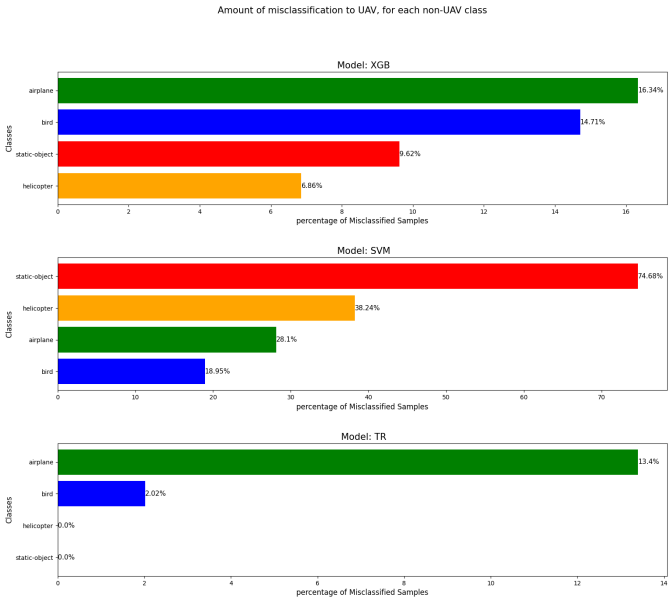

Fig. 7: False positive analysis. TR, XGB and SVM, respectively represent the Transformer, XGBoost and SVM and models. Each graph displays the number of sample misclassified to UAV, for each non-UAV object.

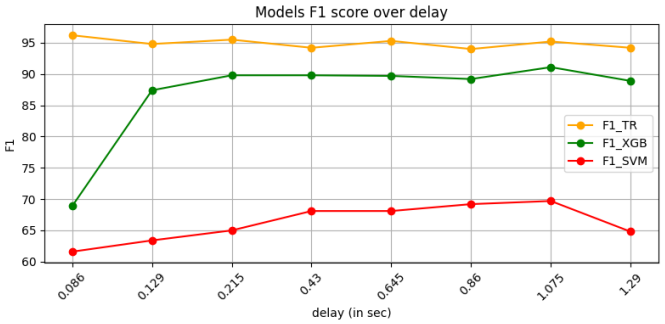

Fig. 8: F1 over delay. F1-TR, F1-XGB and F1-SVM respectively represent F1 of Transformer, XGBoost and SVM models over different delays.

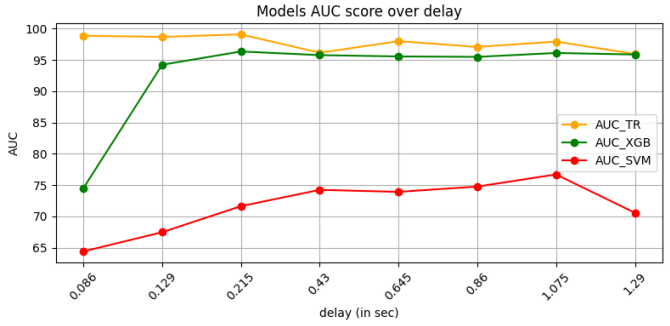

Fig. 9: AUC over delay. AUC-TR, AUC-XGB and AUC-SVM respectively represent AUC of Transformer, XGBoost and SVM models over different delays.

features	node id	f1	f1 imp	precision	recall	size	proportion	min val1	max val1	min val2	max val2
min_x	13	0.93361	0.011296	0.833333	0.962567	266	10.415035	0.320485	0.617281	-1000	inf
min_x-min_y	17	0.932122	0.020575	0.902778	0.939759	498	19.498825	0.320269	inf	-1000	-0.21703
min_x-min_z	16	0.824176	0.025806	0.75419	0.84375	260	10.18011	0.173997	0.908862	-0.56845	1.498546
min_y	20	0.864198	0.003535	0.56	1	25	0.978857	-0.794057	-0.790008	-1000	inf

TABLE III: Transformer 4 bad data slices The table represents 4 data slices, on which the Transformer model performs the worst. Each data slice is represented by the min and max value inside the slice, for each feature. The proportion is the amount of samples, representing by this slice in the total amount of test samples. The table displays the f1 score on the specific slices and the potential improvement of the overall f1 (f1 imp) if these slices had overall f1-score.

train-samples = 5966) data setting. After testing, the FreaAI algorithm [1] , is applied to the predictions to evaluate each model's performance across different data slices. The goal is to assess how XGBoost performs on data slices where the Transformer performs the worst and vice-versa. The algorithm works by partitioning the data into slices using a decision tree based on one or two features, with each slice used to compute specific performance metrics. Once the worst-performing data slice is identified for each model, we evaluate the other model on the same slice to compare its performance. In order to apply the FreaAI algorithm on sequence data we extract for each sequence 6 features, which are the min and the max of each position coordinate.

Table 2 summarizes XGBoost's performance on the worst-performing data slices detected by FreaAI and the potential improvement if predictions on these slices performed well. The worst data slice is identified based on the feature pair f-1-zx/f-1-yx, representing the mean of the difference of slope, between 2 consecutive positions along zx plan and yx plan. This slice contains 145 test samples, within a specific range of values, where the F1 score is 80%. The potential overall improvement for XGBoost on this slice is estimated at 1.2%, while its actual performance stands around 90%. In comparison, when using the same data slice for Transformer prediction, it achieves an F1 score of 93%, outperforming XGBoost by 13%.

Similarly, Table 3 describes the Transformer's performance on its worst-performing data slice. Although the Transformer's overall F1 score is 96%, it drops to 82% on this slice of 260 test samples. When evaluating XGBoost on the same slice, its performance is even lower, with an F1 score of 51%.

V. Discussion

Initially, the experiment demonstrates how XGBoost outperforms SVM on the task. However, when comparing it to the Transformer, we observe that, the Transformer achieves a higher F1 score and AUC than XGBoost in most dataset configuration. Both approaches show strong performance in low-latency detection.

The slice analysis further reveals that the Transformer can successfully handle challenging cases where XGBoost struggles. However, additional slice analysis is needed to fully evaluate the Transformer's consistency, as this may not always hold true across different scenarios.

VI. Conclusions

Overall, both approaches prove to be effective, delivering promising results in the UAV detection task. The Transformer demonstrates exceptional performance, achieving over 95% in accuracy, F1 score, and 99 AUC, consistently outperforming traditional ML algorithms like XGboost, on the optimal data setting (5,2). Additionally, it tends to achieve better results on complex track data. Moreover, applying model-centric optimizations, such as hyperparameter tuning and data augmentation, could further enhance the performance and capabilities of both methods.

References

[1] S. Ackerman, O. Raz, and M. Zalmanovici, FreaAI: Automated extraction of data slices to test machine learning models, in *Proc. Int. Workshop on Engineering Dependable and Secure Machine Learning Systems*, Springer, 2020, pp. 67–83.

[2] W. S. Chen, J. Liu, and J. Li, Classification of UAV and bird target in low-altitude airspace with surveillance radar data, *Aeronautical Journal*, vol. 123, no. 1260, pp. 191–211, 2019.

[3] M. Messina and G. Pinelli, Classification of drones with a surveillance radar signal, in *Proc. Int. Conf. Computer Vision Systems*, Springer, 2019, pp. 723–733.

[4] N. Mohajerin, J. Histon, R. Dizaji, and S. L. Waslander, Feature extraction and radar track classification for detecting UAVs in civilian airspace, in *Proc. 2014 IEEE Radar Conf.*, IEEE, 2014, pp. 0674–0679.

[5] P. Molchanov, R. I. A. Harmanny, J. J. M. de Wit, K. Egiazarian, and J. Astola, Classification of small UAVs and birds by micro-Doppler signatures, *Int. J. Microwave and Wireless Technologies*, vol. 6, no. 3–4, pp. 435–444, 2014.

[6] M. Ritchie, N. Peters, and C. Horne, Radar UAV and bird signature comparisons with micro-Doppler, 2021

[7] D. Tahmoush, UAV discrimination from birds using radar track information, in *Radar Sensor Technology XXVII*, vol. 12535, SPIE, 2023, pp. 257–263.

[8] A. Vaswani, Attention is all you need, in *Advances in Neural Information Processing Systems*, 2017.